Document 02 — APIs, HTTP, JSON & Backend Basics

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

01_python_foundations — you'll reuse two of its scripts here, copied unchanged into `practice/build_task/`:

- `config.py` — **What:** `load_config() -> Config`. **Why:** every later script needs settings loaded once, the same way, instead of each one reading `os.environ` by hand. **How:** copy `01_python_foundations/practice/build_task/config.py` in as-is; don't rewrite it.
- `logging_setup.py` — **What:** `get_logger(name)`. **Why:** every later script logs through the same setup, so DEBUG/ERROR behave consistently everywhere. **How:** copy `01_python_foundations/practice/build_task/logging_setup.py` in as-is; don't rewrite it.

How to Read & Practice This Document

- **What:** talking to other services over the internet (HTTP), in a way that doesn't break under real-world conditions.
- **Why:** every call to an LLM is really just an HTTP call underneath. If you can't handle timeouts and retries here, you won't be able to handle them for OpenAI either — you'll just be confused one layer up instead of here.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercises closed-book. This shows what you actually remember. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open — that's normal. 4. Try the **Build Task** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1 → Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud what your retry code actually does, and why. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Picture Doc01's config and logger already working. Now your program needs to actually go get something from the outside world — call OpenAI, call any API. That's what this whole document is about: the network is unreliable, and your code has to expect that, not be surprised by it.

First, every call is a **request** that gets a **response** — a status code tells you broadly what happened (worked, your fault, their fault), but it doesn't tell you *what to do next*. That's a decision you have to make yourself, based on which kind of failure it was.

Second, a network call can just hang forever if you don't set a **timeout** — the server never replies, and your program never notices, it just waits. So every single outbound call gets a maximum wait time, no exceptions.

Third, some failures are worth trying again (a timeout, a busy server, a rate limit) and some aren't (a bad request is still bad the second time you send it). **Retrying with backoff** is the disciplined way to try again — wait a little longer each time, add some randomness, and give up after a fixed number of tries instead of looping forever.

Fourth, even a response that "worked" (status 200) can still be lying to you — the body might not be valid JSON at all, or it might be valid JSON missing the exact field your code expects. Trusting a 200 blindly is how confusing bugs happen three functions downstream from the actual problem.

That's the story: HTTP basics teach you what a request/response actually promises. Timeouts stop your program from hanging silently. Backoff-with-jitter is how you retry without making things worse. And checking JSON shape (not just parsing it) closes the gap between "the network worked" and "the data is actually what I expected." The **Build Task** at the end turns all four of these into one `http_client.py` that every later document — including Doc04's OpenAI calls — reuses instead of re-solving this from scratch.

Core Concepts (read this first — everything you need is here)

Topics on this page: HTTP: a request gets a response · Timeouts · Retrying with exponential backoff and jitter · JSON: two different kinds of "wrong"

HTTP: a request gets a response

Every API call is a **request** answered by a **response**. The request has a method (`GET`, `POST`...), a URL, headers, and sometimes a body. The response has a status code, headers, and a body (usually JSON). The status code's first digit is the group that matters — it worked, *you* made a mistake, or the *server* had a problem — and your code should branch on that group, not on "200 or not."

How it really works

- **Idempotent** means "doing it twice leaves things exactly like doing it once." `GET`, `PUT`, `DELETE` are idempotent; `POST` usually is not — sending "create order" twice can create two

orders. This is why an automatic retry of a failed `GET` is safe, and a retry of a `POST` needs an idempotency key (many payment APIs support one) or isn't safe at all.

- `requests` follows redirects (3xx) automatically by default — you rarely see them unless you set `allow_redirects=False`.
- Headers are case-insensitive on both sides (`Content-Type` and `content-type` are the same header) — `requests` normalizes this in `response.headers`.
- `response.ok` is `True` for any status under 400 — a fast first check, but it hides the difference between "worked" and "worked, sort of" (a 3xx you didn't expect).
- Not every 5xx deserves a retry: `501 Not Implemented` means the server will *never* support that request — retry only `500`, `502`, `503`, `504`.
- `429` is the one 4xx worth retrying — "you're not wrong, you're just too fast." Use the `Retry-After` header if the server sent one, instead of guessing your own wait.

Code	Group	What it means	What your code should do
`200` / `201`	2xx success	It worked (`201` = created)	Read the body — still check its shape (see the JSON topic)
`400`	4xx, your mistake	Badly formed request	**Don't retry** — fix the request, log the body
`401` / `403`	4xx, your mistake	No valid login / not allowed	**Don't retry** — check your key or permissions
`404`	4xx, your mistake	Doesn't exist	**Don't retry** — check the URL or ID
`429`	4xx, the exception	Rate limited	**Retry after waiting** — use `Retry-After` if present
`500` / `502` / `503` / `504`	5xx, server problem	Server failed or overloaded	**Retry with backoff**, up to a limit

Common mistakes:

- *Mistake:* checking only `if response.status_code == 200:`. → *Symptom:* a `404` (your bug) and a `503` (their bad minute) land in the same `else` — either both get retried or neither does; your logs show the same `400` repeated forever. → *Fix:* branch on the group (4xx / 5xx / 429), not on "200 or not."
- *Mistake:* retrying a `POST` with no idempotency protection. → *Symptom:* a timeout on "create order" that actually succeeded server-side creates a second order on retry. → *Fix:* an `Idempotency-Key` header, same value on every retry of that one request.

Where you'll meet it: every OpenAI call (Doc04) is an HTTP `POST` underneath, and the SDK's error types (`RateLimitError`, `AuthenticationError`) are these status codes with names. A tool that calls a weather or search API (Doc06) is one HTTP request. In Doc12 you're on the *other* side

— your FastAPI server *returns* these codes. In a multi-agent system (Doc11, Project 4), one agent handling status codes badly can stop the whole team.

Quick cheat sheet:

- 2xx = worked, 4xx = your request is wrong, 5xx = the server failed, 429 = slow down.
- Never retry `400` / `401` / `403` / `404`; always consider retrying `429` / `500` / `502` / `503` / `504`.
- `GET` / `PUT` / `DELETE` are safe to repeat; `POST` usually is not.

Timeouts: the failure that never tells you it's happening

A timeout is the maximum time *you* allow a network call to take — when it's up, your own code raises an error instead of waiting forever. `requests` has **no default timeout** — a call with none can hang forever if the server never replies, with no error and no log line.

How it really works

- Two numbers matter: **connect timeout** (how long to wait to even reach the server — keep short, 3-5 s) and **read timeout** (how long to wait *between* pieces of data once connected — not a limit on the whole download, since a slow trickle resets it every time data arrives).
- `timeout=(connect, read)` in `requests`, or one number for both. Fires as `ConnectTimeout` or `ReadTimeout`, both children of `Timeout` — catching `Timeout` catches both.
- LLM calls need a much longer read timeout than normal APIs — a model may think for 30-60+ seconds before the first byte. `OpenAI(timeout=60.0)` applies to every call made with that client.
- A hard *total* time budget (e.g. a user waiting on a web page) isn't the same as `timeout=`. Track your own deadline with `time.monotonic()` if several calls together must fit inside one limit.

Situation	Timeout to use	Why
Normal API call (weather, GitHub)	Short connect, medium read — `(3, 10)`	Usually answers in under a second
Non-streaming LLM call	Short connect, **long** read — `(5, 120)`	Model may think a long time before answering
Streaming LLM call	Short connect, medium read — `(5, 30)`	Read timeout is the gap between chunks, which come often
Health check	Very short both — `(1, 2)`	Its whole job is a fast yes/no
No timeout at all	**Never**	There's no situation where waiting forever is correct

Common mistakes:

- *Mistake:* thinking `timeout=10` limits the *whole* call. → *Symptom:* a call with `timeout=10` shows up in logs taking 40 seconds. → *Fix:* the read timeout resets on every chunk — track your own deadline if you need a true total limit.
- *Mistake:* no timeout at all ("it'll come back eventually"). → *Symptom:* the program just hangs, forever, with nothing in the logs. → *Fix:* always pass `timeout=` — there is no case where its absence is correct.

Where you'll meet it: this document's Build Task puts a default timeout on every call. Doc04's OpenAI client takes one too. Doc08b's async calls need `asyncio.wait_for`. In a multi-agent system (Doc11), one agent waiting forever blocks every agent after it, so each agent's calls get a timeout and the whole run gets a time budget too.

Quick cheat sheet:

- `requests` has no default timeout — always pass one.
- `timeout=(connect, read)`: short connect, read sized for the job.
- Read timeout ≠ total time — track your own deadline for a hard total limit.
- Catch `requests.exceptions.Timeout` to catch both kinds.

Retrying with exponential backoff and jitter

A retry means "the call failed, try again." **Exponential backoff** waits longer after each failure (1s, 2s, 4s, 8s). **Jitter** adds randomness so many clients don't all retry at the same instant. Without both, a struggling server gets hit by a "retry storm" the moment it's already overloaded.

How it really works

- "Full jitter" is the common version: pick a random wait between 0 and the backoff value, capped at a maximum — `random.uniform(0, min(cap, base * 2**attempt))`.
- If `Retry-After` is present on a `429`, use that number exactly instead of guessing your own backoff — the server told you the right answer.
- A fixed attempt limit matters as much as the backoff itself: a truly broken service turns an unlimited retry loop into money burned on a paid API.
- Retrying a `POST` can create a duplicate side effect (a second order, a second email) — only safe with an idempotency key, or after checking whether the first attempt actually landed.
- **Retries stack.** Your own wrapper retrying 3 times, wrapping an SDK that retries 2 more times by default (`OpenAI(max_retries=2)`), wrapping an agent step that retries 3 times again, turns one bad minute into up to 3×3×3 = 27 real calls. Pick exactly one layer to own retries and set the others to 0.

Situation	Retry?	Why
Timeout or connection error	**Yes**, with backoff	May be fine a moment later

Situation	Retry?	Why
`500` / `502` / `503` / `504`	**Yes**, with backoff	Server trouble is often short
`429` with `Retry-After`	**Yes**, exactly that wait	The server told you the right answer
`429` with no header	**Yes**, with backoff	You still need to slow down
`400` / `401` / `403` / `404`	**No**	The request is wrong and stays wrong
A `POST` that creates something	Only with an idempotency key	Otherwise risks a duplicate
Every attempt failed	**Stop**, raise a clear error	Endless retries hide the problem and burn money

Common mistakes:

- *Mistake:* retrying instantly, with no wait. → *Symptom:* an already-overloaded server gets hit harder the moment it fails — a self-made retry storm. → *Fix:* exponential backoff, always.
- *Mistake:* more than one layer owning retries (your code, the SDK, the agent loop). → *Symptom:* the provider's usage page shows far more calls than your own logs. → *Fix:* one layer retries; the others are set to 0.

Where you'll meet it: this document's Build Task (`request_with_retry`) is exactly this loop. Doc07's agent loop hits rate limits quickly. In multi-agent systems (Doc11, Project 4), several agents share one API key and one rate limit — without jitter, they retry together and hit the limit together, again.

Quick cheat sheet:

- Retry: timeouts, connection errors, `429` , `500` / `502` / `503` / `504` . Never: `400` / `401` / `403` / `404` .
- Wait = `random.uniform(0, min(cap, base * 2**attempt))` — full jitter.
- `Retry-After` , if present, beats your own guess.
- Always a fixed attempt limit, then raise.
- Exactly one layer owns retries — check what your SDK already does by default.

JSON: two different kinds of "wrong"

JSON is the plain-text format almost every API uses; `json.loads()` / `response.json()` turn it into Python dicts and lists. A response can be wrong in two very different ways: **(1) invalid JSON** — the body doesn't parse at all (an HTML error page, a cut-off reply) — fails immediately, at `.json()` . **(2) valid JSON, wrong shape** — it parses fine, but a field is missing, `null` , or the wrong type — fails *later*, as a confusing `KeyError` far from the API call.

How it really works

- Status `200` only proves the *transport* worked — it says nothing about whether the *content* is what you expected. A gateway or proxy can return a `200` HTML error page.
- `requests.exceptions.JSONDecodeError` is a child of both `json.JSONDecodeError` and `ValueError` — catching either works.
- `.get(key, default)` is only correct when missing is genuinely normal. For a required field, a quiet default (`data.get("price", 0)`) turns a real problem into a silently wrong answer — check and raise instead.
- For a large or shared shape used in many places, describe it once with a Pydantic model (`Order.model_validate(data)`) instead of scattering manual checks everywhere.
- Text from an LLM that's "supposed to be" JSON can fail *both* ways at once — extra words before the JSON (problem 1) or a renamed/missing field (problem 2) — expect both.

Situation	What to do	Why
Any response you'll parse	Wrap `.json()` in `try/except JSONDecodeError`	An HTML error page with status 200 does happen
A field you **must** have	Check right after parsing, raise a clear error if missing	Fail near the cause, not three functions later
A field that's truly optional	`data.get("key", default)`	Missing is normal here
A required field	**Don't** use `.get()` with a quiet default	Hides a real problem as a fake "empty" value
Text from an LLM that should be JSON	Parse **and** check the shape	Models add extra words, cut off, or rename fields

Common mistakes:

- *Mistake:* `.get(key, default)` "to be safe" on a field that's actually required. → *Symptom:* the API stops sending `price`, nothing crashes, every product silently costs 0 until a customer complains. → *Fix:* for each `.get()` with a default, ask "is missing really normal here?" If not, check and raise.
- *Mistake:* trusting a `200` status without checking the body's shape. → *Symptom:* a strange `KeyError` or `TypeError` several functions away from the actual API call. → *Fix:* check required fields right after parsing, with a message naming the field.

Where you'll meet it: Doc04's structured output is a way to make the model reliably return JSON matching a schema. Doc06's tool arguments arrive as a JSON string, checked with Pydantic — problem 2 again, with a better tool. In RAG (Doc08), document metadata arrives as JSON from many sources with different shapes. In multi-agent systems (Doc11, Project 4), a writer agent that sends a draft with no `title` field should be rejected at the handoff, not crash the next agent two steps later.

Quick cheat sheet:

- Status `200` means the message arrived, not that the content is correct.
- Catch `JSONDecodeError` around `.json()`; check required fields right after.
- `.get(key, default)` only for genuinely optional fields.
- Log a short slice of a bad body (`r.text[:200]`) so you can see what came back.
- For big or shared shapes, use a Pydantic model.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- requests library docs (Quickstart) — the HTTP client you'll use everywhere, before switching to async clients later.
- MDN — HTTP response status codes — know 200/201/400/401/403/404/429/500/503 by heart.
- MDN — HTTP request methods — GET/POST/PUT/PATCH/DELETE, and what "safe to repeat" means.
- Python `json` module docs — `loads` / `dumps`, and where each one can fail.

Practice Exercises

Setup: same venv as Doc01 — if it's not active, `cd 02_apis_http_json && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New package for this document: `pip install requests`.

Where your code lives: all of it under `02_apis_http_json/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — two of them share one file, each in its own labelled section, the same convention as Doc01.

The full file/folder layout, all exercises:

```
practice/
├── api_first_call_practice.py      Basic
├── retry_backoff_practice.py       Intermediate + Failure (two sections)
├── session_reuse_practice.py       Real-world
├── response_validation_practice.py Edge cases
└── build_task/                     Build Task — its own folder
    ├── http_client.py              request_with_retry(...) -> dict
    ├── exceptions.py               MissingConfigError + 2 HTTP errors
    └── test_http_client.py         proves Test Cases, no real calls
```

Why each script exists:

- `api_first_call_practice.py` — sees a bad URL and a bad response are different failure types.
- `retry_backoff_practice.py` — this loop becomes `http_client.py`'s retry logic almost unchanged.
- `session_reuse_practice.py` — one shared `Session` makes a typo'd/missing header impossible.
- `response_validation_practice.py` — a 200 status says nothing about whether the body is what you expected.
- `build_task/http_client.py` — the one wrapper every later document imports for outbound calls.
- `build_task/exceptions.py` — lets callers retry a transient failure but not a permanent one.
- `build_task/test_http_client.py` — proves the Test Cases, without real network calls.

For this document, save your practice code as:

- **Basic** (your first real API call) is its own topic — save it as `practice/api_first_call_practice.py`.
- **Intermediate** (timeouts and retry-with-backoff) and **Failure** (respecting a real rate limit) are both about retrying a failed call correctly — save them together as `practice/retry_backoff_practice.py`, one section per level.
- **Real-world** (one reusable session instead of repeated headers) is its own topic — save it as `practice/session_reuse_practice.py`.
- **Edge cases** (200 doesn't mean safe to trust) is its own topic — save it as `practice/response_validation_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — your first real API call

- **What:** call any free public API with `requests`, print the status code and the parsed JSON, then deliberately break the URL and read the exact error type Python gives you.
- **Why:** you need to see, once, that a bad URL and a bad response are two completely different kinds of failure with different exception types — that distinction is the whole document.
- **How to code it:** `requests.get("https://api.github.com")`, print `.status_code` and `.json()`. Then change the URL to something malformed (like `"http://broken"`) and wrap the call in `try/except` — print the exception's type with `type(e).__name__`.
- **Save as:** `practice/api_first_call_practice.py`.

- **Used later by:** every other exercise on this page calls a real API the same way — this is the shape they all build on.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — timeouts and retry-with-backoff

- **What:** add a timeout to a call, force it to actually fire, then wrap a call in a retry loop with exponential backoff and a limit on attempts.
- **Why:** this exact wrapper is what this document's Build Task turns into `http_client.py` — every project after this one calls through it.
- **How to code it:** `requests.get(url, timeout=(3, 5))` pointed at an address that won't respond (like `http://10.255.255.1`), catch `requests.Timeout`. Then write a `for attempt in range(max_attempts):` loop that retries with `time.sleep(2 ** attempt)` between tries.
- **Save as:** `practice/retry_backoff_practice.py`, under an `# Intermediate` section (this file also holds the Failure exercise below, in its own `# Failure` section).
- **Builds on:** Basic's plain `requests.get()` call — same call, now wrapped in a timeout and a retry loop.
- **Used later by:** the Failure exercise right below (same file, `Retry-After` added on top), and the Build Task, which **copies** this loop into `http_client.py` almost unchanged.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — one reusable session instead of repeated headers

- **What:** wrap login info (a fake bearer token is fine) into one `requests.Session()` object, instead of passing headers to every call by hand.
- **Why:** repeating the same headers dict in five different functions is exactly how one of them ends up with a typo'd or missing header — a session object makes that impossible.
- **How to code it:** `session = requests.Session()`, `session.headers.update({"Authorization": "Bearer fake-token"})`, then make 2-3 calls through `session.get(...)` instead of `requests.get(...)` and confirm the header goes out on all of them.
- **Save as:** `practice/session_reuse_practice.py`.
- **Builds on:** Basic's single `requests.get()` call, now made through a shared `Session` instead of one at a time.
- **Used later by:** the Build Task's `http_client.py`, which holds one `Session` for the whole wrapper instead of a new connection per call.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — 200 doesn't mean safe to trust

- **What:** handle a response with status 200 but a body that isn't valid JSON, and separately, a response with valid JSON but a field your code expects is missing.

- **Why:** a status code only tells you the *transport* succeeded — it says nothing about whether the *content* is what you expected. Mixing up these two is a very common real bug.
- **How to code it:** fake a `Response`-like object (or point at a URL that returns HTML) and call `.json()` inside a `try/except json.JSONDecodeError`. Separately, parse a real JSON dict and access a key that isn't there with `.get("missing_key")` vs. `["missing_key"]` — see the difference in what happens.
- **Save as:** `practice/response_validation_practice.py`.
- **Builds on:** the same `requests.get()` shape from Basic, now checking the *body*, not just the status code.
- **Used later by:** the Build Task's `http_client.py`, which runs this same two-step check on every response before returning it.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — respecting a real rate limit

- **What:** simulate a 429 response and handle it correctly — respecting a `Retry-After` header if the server sent one, falling back to exponential backoff if not.
- **Why:** OpenAI (and most real APIs) rate-limit you, and a client that doesn't back off correctly makes the problem worse for itself and everyone sharing that API key.
- **How to code it:** build a fake response object with `status_code = 429` and a `headers = {"Retry-After": "2"}`. Write a function that checks for that header, sleeps that long if present, or falls back to `2 ** attempt` seconds if not.
- **Save as:** `practice/retry_backoff_practice.py`, under a `# Failure` section (this file also holds the Intermediate exercise above, in its own `# Intermediate` section).
- **Builds on:** the Intermediate retry loop in this same file — same loop, one more branch for the `Retry-After` header.
- **Used later by:** the Build Task's `http_client.py`, which folds this exact `Retry-After` check into its retry logic; every later document that calls an LLM in a loop (Doc07) hits this same rate-limit shape for real.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — HTTP Client Wrapper

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: a small `http_client.py` that every later project will use for outbound calls.

Requirements:

- Wraps `requests` with a default timeout (never call without one).
- Retries temporary failures (timeouts, 5xx, 429) with exponential backoff and jitter, up to a limit.

- Does NOT retry permanent failures — fails right away, with the response body attached to the error. The retryable 4xx codes are `429` (too fast) and `408` (the server itself says "request timeout"); every other 4xx is your mistake and will fail again the same way.
- Logs each attempt at `DEBUG` level, and each final failure at `ERROR` level — **What:** `get_logger(__name__)` from Doc01's `logging_setup.py`. **Why:** one consistent logging setup everywhere, instead of reconfiguring handlers per file. **How:** copy `logging_setup.py` in unchanged, same as Doc02's own `config.py` above.
- **Think about `POST` before you retry it.** A retry is only safe when repeating the call changes nothing (`GET`, `PUT`, `DELETE`). A `POST` that creates something — an order, a payment, an email — can create a **second** one if the first request actually worked and only the *reply* was lost. Either retry `POST` only when the API supports an idempotency key, or make retrying `POST` an option the caller has to switch on, and write down which choice you made and why.

Inputs: a URL, an HTTP method, optional headers/body, optional limits on retries/timeout.

Outputs: the parsed JSON response, or a clear, typed error if it fails.

Constraints: no bare `except:`. Pull the retry-decision logic — classifying a status code, computing the backoff delay, reading `Retry-After` — into small, pure functions you can test directly with plain values, the same way the timeout/retry and rate-limit exercises already did. The retry loop itself gets exercised with a real call, same as those exercises — no fake server, no new testing tool.

Builds on: Intermediate's retry loop, Failure's `Retry-After` handling, Real-world's shared `Session`, and Edge cases's two-step JSON check — **copy** all four into this one file rather than starting from scratch. `exceptions.py` reuses Doc01's named-error pattern, now as `TransientHTTPError` / `PermanentHTTPError`.

Used later by: every later document that calls an outside API imports `http_client.py` instead of calling `requests` directly. Doc04 is the first to say so.

Suggested files:

```
02_apis_http_json/practice/build_task/
├── http_client.py       request_with_retry(method, url, **kwargs) -> dict
├── exceptions.py        MissingConfigError (Doc01) + 2 HTTP errors
├── test_http_client.py  tests the pure decision functions directly
├── config.py            copied from 01_python_foundations, unchanged
└── logging_setup.py     copied from 01_python_foundations, unchanged
```

- `http_client.py` — **What/Why:** the one wrapper every later document imports for outbound calls, instead of calling `requests` directly.
- `exceptions.py` — **What/Why:** lets callers retry a transient failure but fail immediately on a permanent one (a real 4xx bug).

- `test_http_client.py` — **What/Why:** `is_transient_status`, `compute_backoff_delay`, and `decide_wait_seconds` are tested directly with plain values — the same pattern the timeout/retry and rate-limit exercises already used, no fake server needed.
- `config.py` — **What/Why:** `load_config()` — one function every later script calls for settings, instead of reading `os.environ` by hand.
- `logging_setup.py` — **What/Why:** `get_logger(name)` — one consistent logging setup everywhere, instead of reconfiguring handlers per file.

Run it: `cd practice/build_task && python test_http_client.py` — from inside the folder, so `from http_client import request_with_retry` finds the file next to it.

Functions/Components to build:

- `exceptions.py` → `TransientHTTPError`, `PermanentHTTPError` (next to Doc01's `MissingConfigError`, kept unchanged — `config.py` imports it)
- `http_client.py` → `request_with_retry(method, url, **kwargs) -> dict`
- a backoff helper, like `compute_backoff_delay(attempt: int) -> float`

Expected Behavior

- A timeout, a 5xx, a 429 or a 408 → retried up to the limit, then raises `TransientHTTPError`.
- A 400/401/403/404 → raises `PermanentHTTPError` right away, no retry.
- A 200 with bad JSON → raises a clear parsing error, not a raw, uncaught `JSONDecodeError`.
- A `POST` that is not safe to repeat → not retried silently. It either uses an idempotency key, or the caller had to ask for the retry.

Test Cases

Scenario	Expected
`is_transient_status(429)`, `(500)`, `(503)`	`True` — worth retrying
`is_transient_status(404)`, `(401)`	`False` — a real mistake, not worth retrying
`decide_wait_seconds({"Retry-After": "2"}, 0)`	`2.0` — uses the header the server sent
`decide_wait_seconds({}, 3)`	falls back to `compute_backoff_delay(3)`, no header present
`decide_wait_seconds({"Retry-After": "999999999"}, 0)`	`60.0` — capped, never trusts a server's number blindly

Scenario	Expected
`json.loads("<html>error page</html>")`	raises `json.JSONDecodeError`, same check as the Edge cases exercise
A real `GET` to a working endpoint	Returns the parsed JSON dictionary
A real `GET` to an address that always times out	`TransientHTTPError` after `max_attempts`

Break-It / Debug Preview

- A retry loop with no limit (a retry storm that never ends).
- Backoff with no jitter, tested with many "callers" failing at once (they all retry together).
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Repeating a call safely (idempotency) · exponential backoff + jitter · why you never retry a 400 the same way · REST-style APIs · what a 429 requires a good client to do.

Move On When

Given just a stack trace, you can tell whether the failure is your bug or the server's. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**